

RML (Report Markup Language) is ReportLab's own language for specifying the appearance of a printed page, which is converted into PDF by the utility rml2pdf.

These RML samples showcase techniques and features for generating various types of output and are distributed within our commercial package as test cases. Each should be self explanatory and stand alone.

Table of Contents Example

Contents

Chapter 1	1
Chapter 2	3
Chapter 3	4
Last Chapter	5

(The page numbers in this are dynamically calculated)

RML Example 16: Simpletoc



Chapter 1: About this page

About these pages: the code behind this pdf should give an example of how to set up a table of contents.

There are several ways to do tables of contents. We make use of two different features: *names*, which let you refer to other page numbers or bits of text and ensure the references are filled in as the document is built; and *forms*, which let you defer drawing things until later on.

RML is usually generated by another program. We assume this other program can keep track of the headings as it goes through. So, if you have a book with ten chapters, you will have to write out the chapter names ten times in the TOC as well as in each chapter - but RML will sort out the page numbers for you. The source for this example and the comments below should be enough to put it together.

Names and References

The **name**, **getName**, **namedString** and **evalString** tags are all used in combination to set up cross references. You can give a name to a piece of text, and refer to it elsewhere in the document. If we want to refer to, say, the page on which Chapter 3 starts, we would put the following in the heading or some text just after it:

```
Blah blah<namedString id="Chapter3Start"><pageNumber/></namedString> blah
```

The **pageNumber** tag returns the current page number, and the **namedString** one sets up an internal variable which can be referenced ANYWHERE in the document, before or after the definition. To display this page number elsewhere in the document, you would place the following inside some other paragraph or literal string:

```
...see chapter 3 on page <getName id="Chapter3Start"/> for details.
```

You don't have to use names with page numbers - you can use this construct to store any text, such as references to chapter names or figures.

This cross-reference mechanism is the basis of all indexes, tables of contents, cross references and so on. However it has performance implications. RML will reformat the whole document again and again until all the references are **resolved**. This means that if you define names at the beginning of the document and refer to them later, it can work at full speed (as each paragraph's content is fully known when it is drawn). But if you refer forwards, RML makes two - or in some cases three - passes through the document. Your job will be 2-3 times slower. We'll deal with this in a moment.

In real world documents there is another complication. You might have a fancy cover or front matter, and the logical page number 1 used in printing might not actually be page 1. Likewise, you might be doing a batch of customer docs in one RML job. So, in this case we have a more involved expression, and use the **evalString** tag to work out the number we want. In this example we did this by creating a name for the first page after the cover,

```
...<namedString id="page1">
  <evalString default="XXX">
    <pageNumber/>-<getName id="pageZero"/>+1
  </evalString>
</namedString>...
```

This says 'work out the page number of the cover, add 1 and store that in the variable "page1" for future use'.

Forms and Deferred Drawing

Forms (or as Adobe calls them, Form XObjects) are graphics drawn outside of the individual pages in a PDF file, and which may be used more than once or referred to by name. Conveniently, we let you refer to a form early in a document, and define it later. If you fail to define it later on, you will get an error message on generating the PDF. This lets us get around the two-pass problem: we don't actually draw the table of contents until the end, when we know all the named page number references and can do it in one pass. The techniques used here is to "draw" a form called TOC in the cover page template, and define it at the end of the document. As noted above, it's the programmer's job to regener-

RML Example 16: Simpletoc



ate the chapter titles, but the page numbers take care of themselves.

Ignore the following pseudo-bio babble, it's there for padding

Politics of Foramenifera

Globorotalia Menardii coils according to water temperature and prevailing political situations. Preferring alkaline rather than acidic environments they are however intuitively left-wing

Drosophila Melanogaster

How fruit flies changed my life. Blah blah waffle waffle blah blah yawn yawn

RML Example 16: Simpletoc



CHAPTER2

Politics of Foramenifera

Globorotalia Menardii coils according to water temperature and prevailing political situations. Preferring alkaline rather than acidic environments they are intuitively left-wing

Drosophila Melanogaster

How fruit flies changed my life. Blah blah waffle waffle blah blah yawn yawn

RML Example 16: Simpletoc



CHAPTER3

Acrocladium Cuspidatum

A meaningful relationship with Moss

Rattus rattus

So bad they named him twice Blah blah waffle waffle blah blah yawn yawn

RML Example 16: Simpletoc



Last Chapter

Blah blah waffle waffle blah blah yawn yawn..... Blah blah waffle waffle blah blah
yawn yawn..... Blah blah waffle waffle blah blah yawn
yawn.....
Illustration with a border